159302

159.302 Assignment #1

Search Algorithms

8-Puzzle Problem

n.h.reyes@massey.ac.nz

- UC with the Expanded List
- A* with the Strict Expanded List (h=MisplacedTiles)
- A* with the Strict Expanded List (h= Sum of Manhattan distances)

Experiments and Analysis

- Test all algorithms on a set of Initial-Goal state pairs.
- Compare the algorithms' performance.
- Use the given test routines (batch_run mode) to perform the experiments.
- Run the algorithms until:
  - a solution is found/Q turned empty
- Tabulate the results in an Excel file (for submission).

8-Puzzle

Start-up codes for Assignment #1

SEARCH

Start-up Codes: makefile

The start-up system is comprised of multiple files.

Executable filename

```
Search.exe              : Main.o graphics.o puzzle.o algorithm.o
    g++ -std=c++11 -o Search.exe Main.o graphics.o puzzle.o algorithm.o -l gdi32   -
    static-libgcc -static-libstdc++

Main.o      : Main.cpp graphics.h puzzle.h algorithm.h
    g++ -std=c++11 -c  -Wno-write-strings Main.cpp

puzzle.o : puzzle.cpp puzzle.h
    g++ -std=c++11 -c  -Wno-write-strings  puzzle.cpp

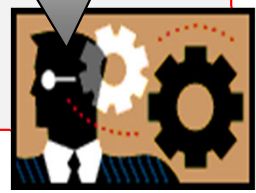
algorithm.o             : algorithm.cpp algorithm.h
    g++ -std=c++11 -c  -Wno-write-strings algorithm.cpp

graphics.o  : graphics.cpp graphics.h
    g++ -std=c++11 -c  -Wno-write-strings graphics.cpp

clean:
    del *.o
    del *.exe
```

Must be preceded by a **tab**.

The program requires that you "**build**" it using a **makefile**.



back

SEARCH

Start-up Codes: makefile

```
Search.exe              : Main.o graphics.o puzzle.o algorithm.o
    g++ -std=c++11 -o Search.exe Main.o graphics.o puzzle.o algorithm.o -l gdi32  -
    static-libgcc -static-libstdc++

Main.o      : Main.cpp graphics.h puzzle.h algorithm.h
    g++ -std=c++11 -c  -Wno-write-strings Main.cpp

puzzle.o : puzzle.cpp puzzle.h
    g++ -std=c++11 -c  -Wno-write-strings  puzzle.cpp

algorithm.o             : algorithm.cpp algorithm.h
    g++ -std=c++11 -c  -Wno-write-strings algorithm.cpp

graphics.o  : graphics.cpp graphics.h
    g++ -std=c++11 -c  -Wno-write-strings graphics.cpp

clean:
    del *.o
    del *.exe
```

The program uses a simple graphics library (**graphics.cpp and graphics.h**) and links to the built-in graphics device interface (**gdi32**) of the Windows operating system. Because of this, the program only compiles in the Windows environment.



SEARCH

Start-up Codes: algorithm implementation

- Place your algorithm implementations inside **algorithm.cpp**
- Include the function prototypes inside **algorithm.h (already pre-defined)**

algorithm.cpp

```
#include "algorithm.h"
using namespace std;

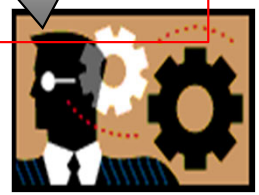
////////////////////////////////////////////////////////////////////////////////////
// Search Algorithm:  A* using the Strict Expanded List
////////////////////////////////////////////////////////////////////////////////////
string aStar_ExpandedList (string const initialState, string const goalState, int &pathLength,
int &numOfStateExpansions, int& maxQLength, float &actualRunningTime,
int &numOfDeletionsFromMiddleOfHeap, int &numOfLocalLoopsAvoided,
int &numOfAttemptedNodeReExpansions, heuristicFunction heuristic ) {

    // put your implementation codes here…

}
```

Please don't change the function names and their formal parameter definitions as the program refers to them when batch runs are executed to perform the experiments.

Place your algorithm implementations inside these files.



- Skeleton functions are provided in the start-up codes.

back

SEARCH

Start-up Codes: function prototypes for algorithms

algorithm.h

```
#ifndef __ALGORITHM_H__
#define __ALGORITHM_H__

#include <ctime>
#include <string>
#include <iostream>
#include <algorithm>
#include <cstdlib>
#include <cstring>
#include <vector>

#include "puzzle.h" //optional only, may be removed


////////////////////////////////////////////////

//Function prototypes

string uc_explist(string const initialState, string const goalState, int& pathLength, int &numOfStateExpansions, int& maxQLength,
                  float &actualRunningTime, int &numOfDeletionsFromMiddleOfHeap, int &numOfLocalLoopsAvoided, int &numOfAttemptedNodeReExpansions);


string aStar_ExpandedList(string const initialState, string const goalState, int& pathLength, int &numOfStateExpansions, int& maxQLength,
                  float &actualRunningTime, int &numOfDeletionsFromMiddleOfHeap, int &numOfLocalLoopsAvoided, int &numOfAttemptedNodeReExpansions, heuristicFunction heuristic);



#endif
```

function name

Formal parameters

Feel free to add more variables, functions, classes, etc. inside **algorithm.cpp** and **algorithm.h**. However, you are not allowed to change the names and formal parameters of the algorithms. The batch experiment routines refer to them to automate the execution of the experiments.



back

SEARCH

Main function (arguments)

The program requires some arguments.

```
int main( int argc, char* argv[] ){

  string path;


  if(argc < 3){
      cout << "<< SEARCH ALGORITHMS >>" << endl;
      cout << "please include missing parameters." << endl;
      cout << "SYNTAX #1: search.exe <TYPE_OF_RUN = \"batch_run\" or \"single_run\" or \"animate_run\"> ALGORITHM_NAME \"INITIAL STATE\" \"GOAL STATE\" " << endl;
      cout << "SYNTAX #2: search.exe <TYPE_OF_RUN = \"batch_run\"> ALGORITHM_NAME" << endl;
      exit( Code: 0);
  }
```

Calling the program:

search.exe **type_of_run** [algorithm] [start_state] [goal_state]

Mode of execution

The main function parameters allow the program to switch between a number of modes of execution.

back

SEARCH

Main function (arguments)

The program requires some arguments.

```
int main( int argc, char* argv[] ){

   string path;


    if(argc < 3){
        cout << "<< SEARCH ALGORITHMS >>" << endl;
        cout << "please include missing parameters." << endl;
        cout << "SYNTAX #1: search.exe <TYPE_OF_RUN = \"batch_run\" or \"single_run\" or \"animate_run\"> ALGORITHM_NAME \"INITIAL STATE\" \"GOAL STATE\" " << endl;
        cout << "SYNTAX #2: search.exe <TYPE_OF_RUN = \"batch_run\"> ALGORITHM_NAME" << endl;
        exit( Code: 0);
    }
```

search.exe **type_of_run** [algorithm] [start_state] [goal_state]

Mode of execution

1. single_run	run a single algorithm, and show results in text mode
2. animate_run	run a single algorithm, and animate solution
3. batch_run	run a one or more algorithms in text mode

SEARCH

Main function (arguments)

The program requires some arguments.

```
int main( int argc, char* argv[] ){

  string path;


  if(argc < 3){
      cout << "<< SEARCH ALGORITHMS >>" << endl;
      cout << "please include missing parameters." << endl;
      cout << "SYNTAX #1: search.exe <TYPE_OF_RUN = \"batch_run\" or \"single_run\" or \"animate_run\"> ALGORITHM_NAME \"INITIAL STATE\" \"GOAL STATE\" " << endl;
      cout << "SYNTAX #2: search.exe <TYPE_OF_RUN = \"batch_run\"> ALGORITHM_NAME" << endl;
      exit( Code: 0);
  }
```

As an example, we can use the **animate_run** parameter setting, to instruct the program to apply the A* algorithm (with the strict expanded list, and Sum of Manhattan distances as heuristic) to solve a given problem instance of the puzzle. Once the solution is found, it will display an animation of how the puzzle is solved.

C:\> search.exe **"animate_run"** astar_explist_manhattan "608435127" "123456780"

back

SEARCH

Main function (arguments)

The program requires some arguments.

```
int main( int argc, char* argv[] ){

  string path;


  if(argc < 3){
      cout << "<< SEARCH ALGORITHMS >>" << endl;
      cout << "please include missing parameters." << endl;
      cout << "SYNTAX #1: search.exe <TYPE_OF_RUN = \"batch_run\" or \"single_run\" or \"animate_run\"> ALGORITHM_NAME \"INITIAL STATE\" \"GOAL STATE\" " << endl;
      cout << "SYNTAX #2: search.exe <TYPE_OF_RUN = \"batch_run\"> ALGORITHM_NAME" << endl;
      exit( Code: 0);
  }
```

Here are some examples of how we can run the program.

search.exe “**single_run**" “**astar_explist_manhattan**" "042158367" "012345678"

search.exe “**animate_run**" “**astar_explist_manhattan**" "042158367" "012345678"

search.exe “**batch_run**" “**astar_explist_manhattan**"

search.exe “**batch_run**" “**all**"

back

How to run the program using the **single_run** mode?

single_run <algorithm>

Run an algorithm using a single pair of (start-goal) combination

back

SEARCH

Main function (required arguments)

Running the program:

Example:

INITIAL STATE GOAL STATE

search.exe “**single_run**" “**astar_explist_manhattan**" "042158367" "012345678"

type of run = "batch_run" or "single_run“ or “animate_run"

Algorithm name

For testing a single algorithm on a single pair of (start-goal) pair

Algorithm name (you are not allowed to change these names)

“uc_explist"
"astar_explist_manhattan"
"astar_explist_misplacedtiles"



back

How to run the program using the **animate_run** mode?

animate_run <algorithm>

Run an algorithm using a single pair of (start-goal) combination, and see the animation of solution.



back

SEARCH

Main function (required arguments)

Running the program:

Example:

Algorithm name

INITIAL STATE

GOAL STATE

search.exe “**animate_run**" “**astar_explist_manhattan**" "042158367" "012345678"

type of run = "batch_run" or "single_run“ or “animate_run"

Algorithm name (you are not allowed to change these names)
“uc_explist"
"astar_explist_manhattan"
"astar_explist_misplacedtiles"

For testing a single algorithm on a single pair of (start-goal) pair



back

SEARCH

Animated Solution (run_one.bat)

Using graphics to show the animated solution

```
if (algorithmSelected == "uc_explist") {

path = uc_explist(initialState, goalState, pathLength, numOfStateExpansions, maxQLength,
actualRunningTime, numOfDeletionsFromMiddleOfHeap, numOfLocalLoopsAvoided,
numOfAttemptedNodeReExpansions);

}
...and so on, and so forth...
```

```
if(typeOfRun == "animate_run"){
    if (pathLength == 0) cout << "\n\n*---- NO SOLUTION fo                    endl;

    if (path != "") {
        AnimateSolution(initialState, goalState, path);
    }
}
```

The “animate_run” parameter invokes the AnimateSolution function.

An animation function is provided for you.



e.g. path=“URRDLURRDLR”

Skeleton function

back

SEARCH

Animated Solution (animate_run)

astar_explist_manhattan

```
==========<< EXPERIMENT RESULTS >>======

          Initial State:    608435127
             Goal State:    123456780

            Path Length:           23
 Num Of State Expansions:          662
           Max Q Length:          392
     Actual Running Time:     0.190000
Num of Deletions from MiddleOfHeap:            9
Num of Attempted Node ReExpansions:          683
==========================================================================


press any key to start.
```




```
./search  "animate_run" astar_explist_manhattan "608435127" "123456780"
```

Try **animate_run**



back

How to run the program using the **batch_run** mode?

batch_run <algorithm>

Run an algorithm using a set of (start-goal) state combinations

back

SEARCH

Main function (required arguments)

Algorithm name (you are not allowed to change these names)

"uc_explist"
"astar_explist_manhattan"
"astar_explist_misplacedtiles"

Running the program:

Algorithm name

Example:

search.exe "**batch_run**" "**astar_explist_manhattan**"

type of run = "batch_run" or "single_run" or "animate_run"

Batch runs are provided to run multiple experiments in one call to the program.

There is a **predefined set of puzzle problems** that will be used in the batch run:

```
string list_of_initialStates[5] = { "120483765","208135467","704851632","536407182","638541720" };
string goalState = { "123456780" };
```



back

How to run the program using the **batch_run all** mode?

batch_run <all>

- Run multiple algorithms using a set of (start-goal) state combinations.
- We will use this program feature for generating the complete experiment results for submission.

SEARCH

Main function (required arguments)

Algorithm name (you are not allowed to change these names)

"uc_explist"
"astar_explist_manhattan"
"astar_explist_misplacedtiles"

Running the program:

All algorithms

Example:

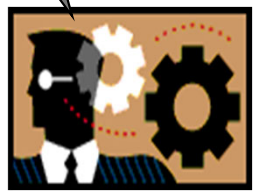
search.exe "**batch_run**" "**all**"

type of run = "batch_run" or "single_run" or "animate_run"

batch_runs all will run all the experiments required for testing all the algorithms in your program.

There is a **predefined set of puzzle problems** that will be used in the batch run:

```
string list_of_initialStates[5] = { "120483765","208135467","704851632","536407182","638541720" };
string goalState = { "123456780" };
```



back

SEARCH

batch_run all

Running the algorithms

Batch run

- This type of run is used for running all the experiments required for all the algorithms.
- Results are tabulated nicely, according to the requirements of the assignment.

Example:

```
C:\159302\A1>search.exe batch_run all
ALGORITHM,              INIT_STATE,          GOAL_STATE,      PATH_LENGTH,     STATE_EXPANSIONS,  MAX_QLENGTH,  RUNNING_TIME,  DELETIONS_MIDDLE_HEAP, LOCAL_LOOPS_AVOIDED, ATTEMPTED_REEXPANSIONS,  PATH
  uniform_cost_search         ,120483765         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,208135467         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,704851632         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,536407182         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,638541720         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
astar_misplacedtiles          ,120483765         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
astar_misplacedtiles          ,208135467         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
astar_misplacedtiles          ,704851632         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
astar_misplacedtiles          ,536407182         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
astar_misplacedtiles          ,638541720         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
     astar_manhattan          ,120483765         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
     astar_manhattan          ,208135467         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
     astar_manhattan          ,704851632         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
     astar_manhattan          ,536407182         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
     astar_manhattan          ,638541720         ,123456780         ,21          ,17              ,115           ,0.000000       ,0                    ,0                   ,0                      ,DDRRLLLUUURDLUDURDLUU
```

Extra information to help with algorithm implementation analysis.

Convert the results from text to Excel file format. (see template.xlsx)

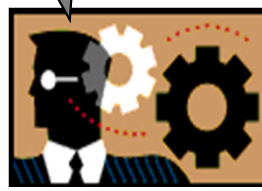
back

Tabulation of Experiment Results

```
C:\159302\A1>search.exe batch_run all
ALGORITHM,              INIT_STATE,          GOAL_STATE,      PATH_LENGTH,     STATE_EXPANSIONS,  MAX_QLENGTH,  RUNNING_TIME,  DELETIONS_MIDDLE_HEAP, LOCAL_LOOPS_AVOIDED, ATTEMPTED_REEXPANSIONS,   PATH
  uniform_cost_search         ,120483765         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,208135467         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,704851632         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,536407182         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
  uniform_cost_search         ,638541720         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
 astar_misplacedtiles         ,120483765         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
 astar_misplacedtiles         ,208135467         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
 astar_misplacedtiles         ,704851632         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
 astar_misplacedtiles         ,536407182         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
 astar_misplacedtiles         ,638541720         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
      astar_manhattan         ,120483765         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
      astar_manhattan         ,208135467         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
      astar_manhattan         ,704851632         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
      astar_manhattan         ,536407182         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
      astar_manhattan         ,638541720         ,123456780         ,21          ,17          ,115          ,0.000000          ,0                  ,0                  ,0                  ,DDRRLLLUUURDLUDURDLUU
```

Sample only

Your assignment will be marked based on the results of your experiments.



Tabulation of Experiment Results

Use your surname_forename.xlsx as the name of your Excel file

Sample only

	A	B	C	D	E	F	G	H	I	J	K
1	ALGORITHM	INIT_STATE	GOAL_STATE	PATH_LENGTH	STATE_EXPANSIONS	MAX_QLENGTH	RUNNING_TIME	DELETIONS_MIDDLE_HEAP	LOCAL_LOOPS_AVOIDED	ATTEMPTED_REEXPANSIONS	PATH
2	uniform_cost_search	120483765	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
3	uniform_cost_search	208135467	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
4	uniform_cost_search	704851632	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
5	uniform_cost_search	536407182	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
6	uniform_cost_search	638541720	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
7	astar_misplacedtiles	120483765	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
8	astar_misplacedtiles	208135467	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
9	astar_misplacedtiles	704851632	123456780	21	143	144	0	0	0	0	DDRRLLLUUURDLUDURDLUU
10	astar_misplacedtiles	536407182	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
11	astar_misplacedtiles	638541720	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
12	astar_manhattan	120483765	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
13	astar_manhattan	208135467	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
14	astar_manhattan	704851632	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
15	astar_manhattan	536407182	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
16	astar_manhattan	638541720	123456780	21	123	148	0	0	0	0	DDRRLLLUUURDLUDURDLUU
17											

results

Your assignment will be marked based on the results of your experiments.



The worksheet must be named "results"

Results must be replicable using your submitted codes.

Sample parameter settings

1. search "single_run" astar_explist_manhattan "608435127" "123456780"
2. search "animate_run" astar_explist_manhattan "608435127" "123456780"
3. search "batch_run" astar_explist_manhattan
4. search "batch_run" astar_explist_misplacedtiles
5. search "batch_run" uc_explist
6. search "batch_run" all

back

S

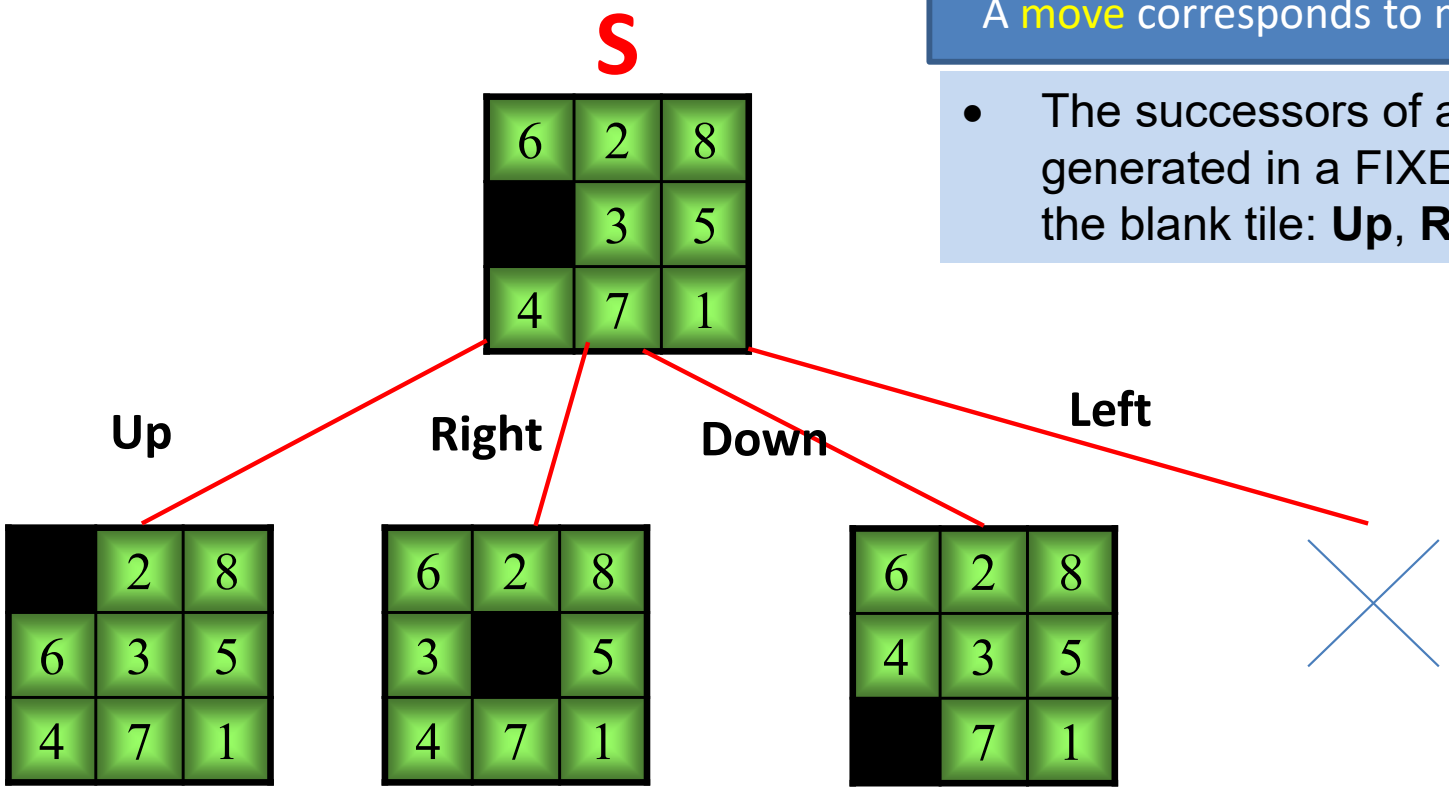
6	2	8
	3	5
4	7	1

The states are the board configurations

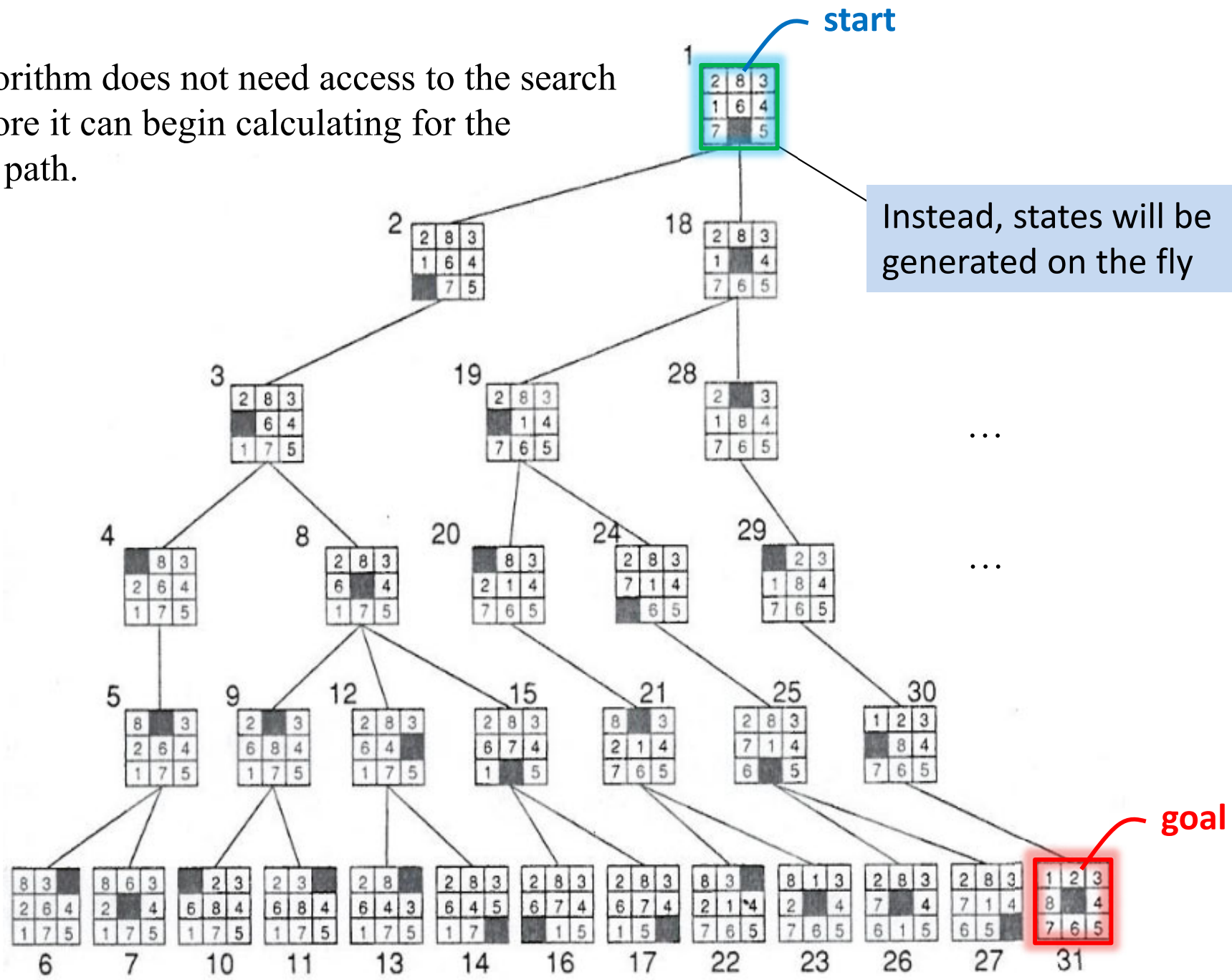
- There are 9! states (362,880 states) but not all of them are reachable from a given board position.

A move corresponds to moving the blank '0'-tile

- The successors of a state are to be generated in a FIXED order, namely move the blank tile: **Up**, **Right**, **Down**, then **Left**.



The algorithm does not need access to the search tree before it can begin calculating for the shortest path.



Example: **Using Depth-limited Depth-First Search**

Example: Depth-limited Depth-First Search

8 PUZZLE

DFS (depth bound = 5)

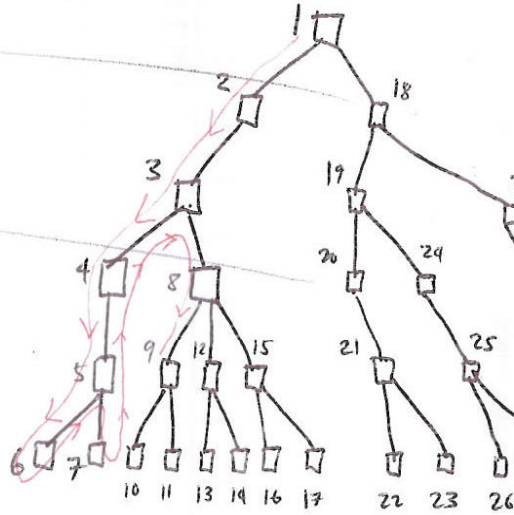
(CLOSED LIST)

Step	Dequeued	Enqueued	Visited List
0			
1		left, up, right	
d = 1			
2		up	
3		up, right, down	
d = 2			
4		right, down	
d = 3			
5		left, right, down	
d = 4			
6			
d = 5 no more expansions!			
7			
d = 5 !			
8			

Successor generator: Left, Up, Right, Down

Store each state as a triple:

- STATE
- PARENT
- LENGTH OF PATH



class Puzzle (Optional – you may choose not to use this)

In the start-up codes, I am providing a class named Puzzle to give the class some clue on how to approach the problem using an object-oriented programming style.

```
class Puzzle{

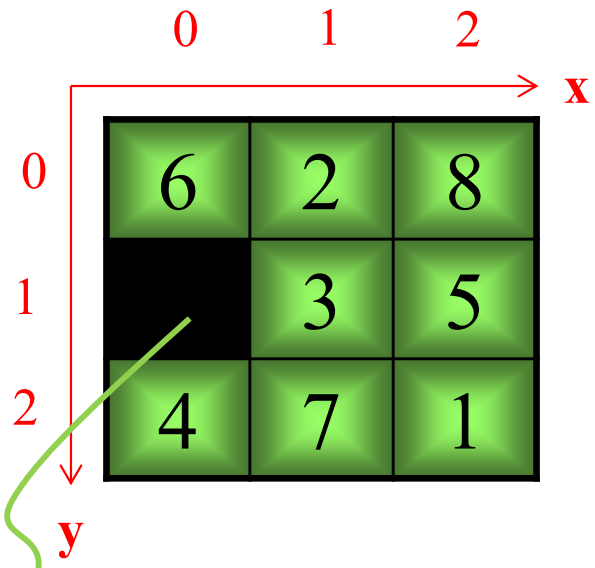
private:

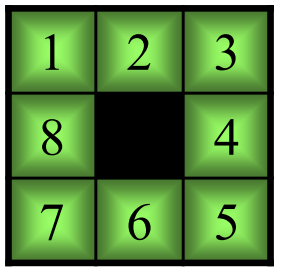
    string path; //moves generated (e.g. URDL..)
    int pathLength;
    int hCost;
    int fCost;          //used by A*
    int depth; //may be used by PDS

    int goalBoard[3][3]; //goal board configuration

    int x0, y0; //coordinates of the blank or 0-tile

    int board[3][3]; //current board configuration
```





Goal

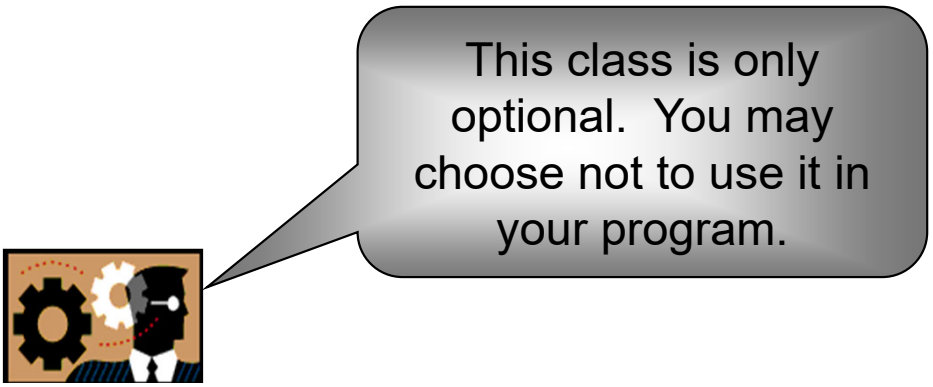
Feel free to add more variables and functions that you may require.

class Puzzle (Optional – you may choose not to use this)

The class **Puzzle** is not complete, if you intend to use it in your assignment, you will have to fill-in the missing functions.

```
void Puzzle::updateFCost(){
    //fCost = ?
}
```

Note that it is not required to use the class Puzzle when implementing the different algorithms - you may design your own class from scratch.




```
//Heuristic function implementation
int Puzzle::h(heuristicFunction hFunction){

 int sum=0;
 int h=0;
 int numOfMisplacedTiles=0;

  switch(hFunction){
      case misplacedTiles:
              //place your implementation here
              //h = ??? //numOfMisplacedTiles;

              break;
      case manhattanDistance:
              //place your implementation here
             //h = ??? //sum of manhattan distance;

             break;
      };
      return h;
}
```

```
Puzzle(const Puzzle &p); //constructor; creates a copy of a given puzzle
Puzzle(string const elements, string const goal); //constructor; generates 2D-array versions corresponding to an initial board configuration and a goal configuration.

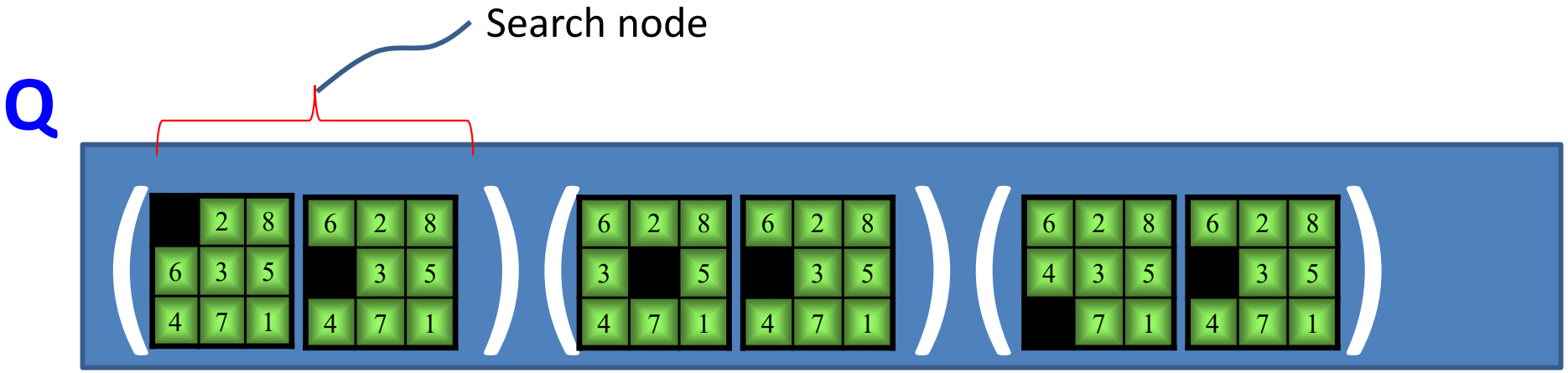
void printBoard(); //prints the content of a puzzle
int h(heuristicFunction hFunction); //computes for the heuristic value of a state
void updateFCost(); //computes for the f-cost of a search node
bool goalMatch(); //returns true if the current board configuration matches the goal state; otherwise, false

bool canMoveLeft();
bool canMoveRight();
bool canMoveUp();
bool canMoveDown();
```

//checks to see if the blank **0-tile** could be moved in a specific direction (codes provided)

```
Puzzle * moveUp();
Puzzle * moveRight();
Puzzle * moveDown()
Puzzle * moveLeft();
```

//creates an instance of a Puzzle (including memory allocation), representing the result of the move taken. It also keeps track of the sequence of moves taken (or the partial path). The function returns a Puzzle object. (codes provided)



- a **Q** is a container for the partial paths (**search nodes**) being considered for expansion.
- depending on the algorithm you are implementing, you may be needing a Queue, Stack or a Priority Queue to implement this **Q** container.

When extending a node, moveUp(), moveRight(), moveDown(), moveLeft() comes handy.

Within a search node, there shouldn't be any repeating nodes/no local loops allowed (i.e. you may have to add variables and test functions to do this, depending on the algorithm you are implementing).

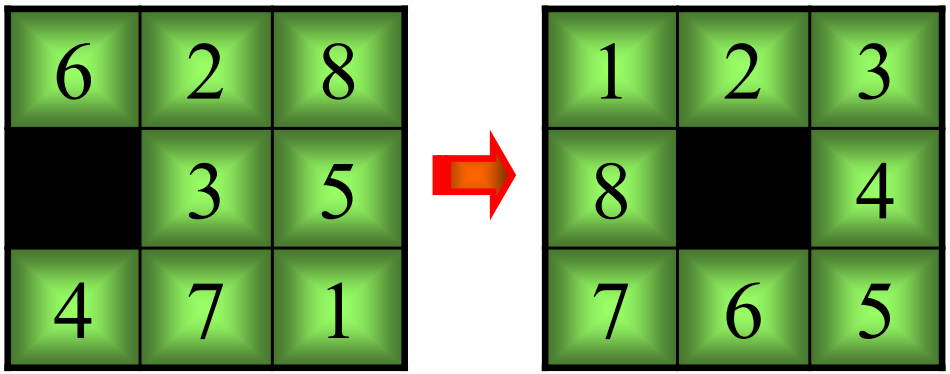
UC and the A* Algorithm

- Implement the Q container using the **heap data structure implementation** available in the C++ Standard Template Library (STL).
- You can convert a **vector** into a **heap** using: make_heap(), push_heap(), pop_heap(), etc.
- Use a custom-built comparator in conjunction with all these heap functions.

SEARCH

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.



S G

Alternative underestimates of "distance" (number of moves) to goal:

1. Number of **misplaced tiles** (7 in the example above).

2. Sum of **Manhattan Distance** of tile to its goal location (17 in example above).
   Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.
   Each move can only decrease the distance of exactly one tile.

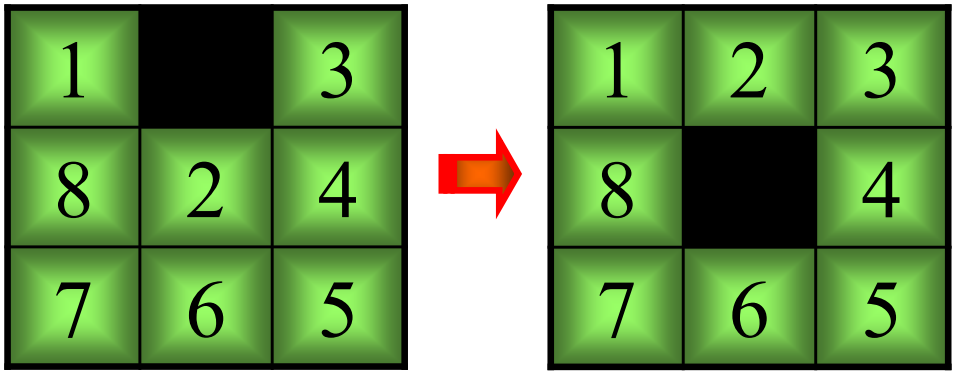
The second of these is much better at predicting actual number of moves.

SEARCH

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.

1. Number of **misplaced tiles**



S **G**

Are we supposed to include the blank tile in the counting of misplaced tiles?

No.

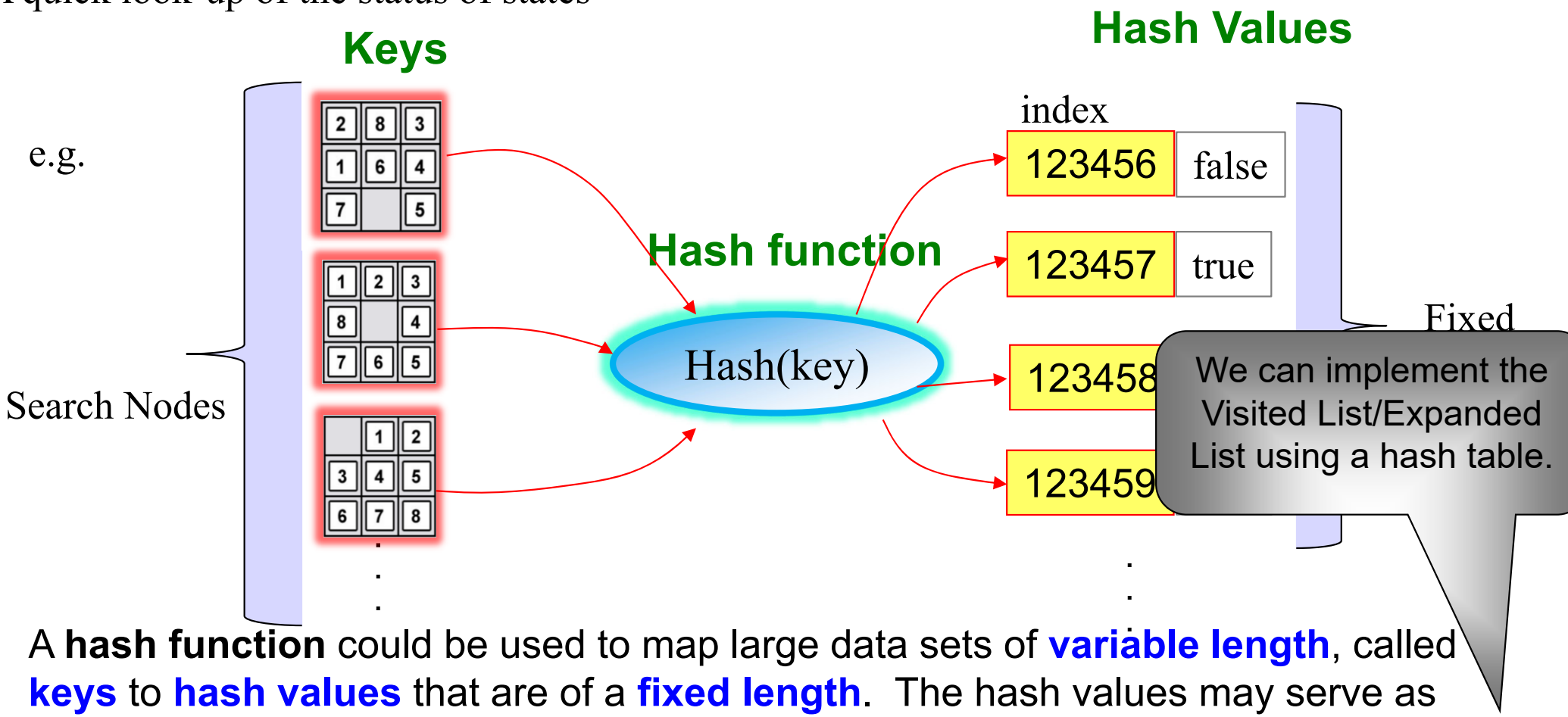
What will be the consequence if we include it? Examine the case above.

The heuristic will no longer be admissible. A* will miss finding the optimal solution if the heuristic is not admissible.

SEARCH

Hash function

A quick look-up of the status of states



A **hash function** could be used to map large data sets of **variable length**, called **keys** to **hash values** that are of a **fixed length**. The hash values may serve as indexes to an associative array.

However, hash values may have duplicates; therefore, may cause collision between keys.



SEARCH

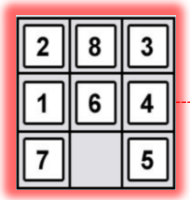
Hash function

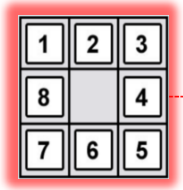
Visited List / Expanded List

A quick look-up of the status of states

Keys **Hash function** **Hash Values** Fixed length

e.g.





Search Nodes

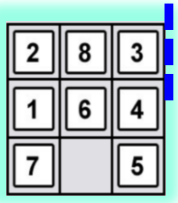


Hash()

Hash Values	
000001	false
362879	true
200000	true
300879	true
. . .	
362880	true

Given a state, we can quickly check whether or not we have visited/expanded it previously.

STATE



Running and Documenting the Experiments

How to save the experiment results to an Excel worksheet?

Checklist

Please accomplish the following checklist to allow for accurate marking of your assignment.

Checklist				
	Item	**your assignment details**		**Comments**
1	Names and ID numbers of Group Members			(maximum of 3 members in a group)
2	Operating System used for testing your codes			e.g. Windows 10/11, MacOS, Ubuntu Linux
3	Compiler used			Only **gcc 13.1** or **clang 14.0** is allowed
4	IDE used			(e.g. SublimeText, Visual Studio)
5	Complete source codes and support files for compilation/linking/execution	***.cpp** *files*	Yes/No	You must submit the complete source codes and support files for compilation purposes. Moreover, your program must compile using the makefile provided.
		***.h** *files*	Yes/No	
		makefile	Yes/No (this is required)	
		tasks.json	Yes (if using Visual Studio Code) / No	
6	Algorithms	uc_explist	full/partial/none	Indicate '**full**', if you have completed the implementation of an algorithm, or 'partial', if you are only submitting a partial implementation.
		astar_explist_manhattan	full/partial/none	
		astar_explist_misplacedtiles	full/partial/none	
7	Is your program able to run using the **batch_run all** mode provided?	Yes/No		Your program must be able to execute in **batch_run all** mode. (Indicate '**Yes**' or '**No**')
8	Experiment Results in Excel Worksheet	Yes/No (Note: Use your **surname_forename.xlsx** as the name of your Excel file. The name of the worksheet must be changed to **results.**)		• indicate '**Yes**' or '**No**' • you will lose **50%** of your grade if you fail to submit this Excel file
9	(Maximum of 2 Bonus marks) for Fast and correct execution of all experiments	Yes/No. (If Yes, list down all enhancements/optimisations you have added.)		a maximum of 2 bonus marks will be awarded if your program can perform all the required experiments correctly (i.e. search.exe batch_run all) in less than 1 minute. Write the total execution time.
		Total time of execution of all	_____ (in seconds)	